

# Task 2 — Local RAG With a Real Evaluation Harness

|               |                                                                              |
|---------------|------------------------------------------------------------------------------|
| TRACK         | Agentic AI — Intern Programme, Task 2 of 3                                   |
| PACKAGE       | <code>packages/rag</code>                                                    |
| PREREQUISITES | TypeScript, SQL basics. Independent of Task 1 — can be done in either order. |
| COST          | Zero. Everything runs locally.                                               |

## Why this task exists

Retrieval-augmented generation is easy to build and hard to build *well*. A weekend version works on the demo query and falls apart on the ninth. The difference between a toy and a system is not the retrieval code — it is having a number that tells you whether a change actually helped.

So the real subject of this task is **measurement**. Retrieval is merely the thing being measured.

Most people who claim to have "improved" a RAG pipeline cannot tell you which change moved which metric, because they never had metrics. They changed five things, the demo felt better, and they shipped it. You are going to do the opposite: change one thing at a time and produce a table.

That habit is worth more than any specific retrieval technique you will learn here, because the techniques will be obsolete in eighteen months and the habit will not.

## Setup

### Runtime

| COMPONENT     | CHOICE                                                       | WHY                                              |
|---------------|--------------------------------------------------------------|--------------------------------------------------|
| Node          | 20 or newer                                                  | Native <code>fetch</code>                        |
| TypeScript    | 5.x, <code>strict: true</code>                               | Non-negotiable                                   |
| Test runner   | Vitest                                                       | Fast, ESM-native                                 |
| Model runtime | Ollama                                                       | One binary, no Python environment                |
| Store         | <code>better-sqlite3</code> + <code>sqlite-vec</code> + FTS5 | Vector and keyword search in one file, no server |

### Models

```
ollama pull qwen2.5:7b-instruct    # ~4.7 GB — generation and judging
ollama pull nomic-embed-text      # ~275 MB — embeddings
```

If the 7b model runs slowly for you, substitute `qwen2.5:3b-instruct` for generation. Keep `nomic-embed-text` either way — swapping the embedding model invalidates every metric you have already recorded.

## Corpus

Clone a mid-size open-source Next.js project — a few hundred files, mixed `.ts`, `.tsx`, and `.md`.

Something real, with genuine naming inconsistency and dead code in it. **Not** a curated tutorial repo. Tutorial repos are internally consistent, which makes retrieval artificially easy and your metrics meaningless.

## Explicitly forbidden

LangChain, LlamaIndex, and any pre-built RAG pipeline. You may read their source. You may not import them.

**Allowed and encouraged:** `better-sqlite3`, `sqlite-vec`, `ts-morph`, `gray-matter`, `zod`, `commander`.

---

## What you are building

A CLI that ingests a real codebase and answers questions about it with file-and-line citations.

```
pnpm rag ingest ./corpus/some-nextjs-repo
pnpm rag ask "How does the middleware handle authentication redirects?"
pnpm rag eval                # runs the golden set, prints metrics
pnpm rag eval --compare baseline.json
```

---

## Requirements

### 1. AST-aware chunking

This is the highest-leverage decision in the task, and the one most people get wrong.

Naive chunking splits every 512 characters. It cuts functions in half. **Half a function is worse than useless as a retrieval unit** — it retrieves, it looks relevant, and it actively misleads the generation step.

Use `ts-morph`. Chunk on semantic boundaries: function declarations, class methods, exported consts, type and interface definitions.

Every chunk carries metadata:

```
interface Chunk {
  id: string;
  content: string;
  path: string;
  startLine: number;
  endLine: number;
  symbolName?: string; // 'handleAuthRedirect'
  kind: 'function' | 'class' | 'type' | 'markdown-section' | 'other';
  contentHash: string;
}
```

Markdown chunks on heading boundaries, carrying the full heading path — `Getting Started > Installation > Docker` — as a prefix on the embedded text. That prefix measurably improves embedding quality: a section titled *"Configuration"* is ambiguous alone and unambiguous with its ancestry attached.

Two cases to handle deliberately:

- **A 600-line function.** Split it, with overlap, and mark the parts as siblings.
- **A 3-line barrel export.** Too small to embed usefully. Merge adjacent small chunks up to a floor of roughly 100 tokens.

## 2. Hybrid retrieval with Reciprocal Rank Fusion

Build two indices over the same chunks.

**Vector** — `nomic-embed-text` via Ollama, stored in `sqlite-vec`. Catches semantic matches: *"how do we check if a user is logged in"* → `verifySession()`.

**BM25** — SQLite FTS5. Catches exact matches: a specific error string, a config key, an unusual identifier.

Each is bad at what the other is good at. Vector search alone will not reliably find `NEXT_PUBLIC_SANITY_PROJECT_ID`. Keyword search alone will not connect *"logged in"* to *"session"*.

Fuse them with Reciprocal Rank Fusion:

$$\text{score}(d) = \sum \text{over rankers } r \text{ of } 1 / (k + \text{rank}_r(d)) \quad \text{where } k = 60$$

**You must implement RRF specifically — not a weighted average of the two scores — and explain why in**

**NOTES.md**. The reason is worth internalising: cosine similarity lives in roughly [0,1] with a compressed useful range, while BM25 is unbounded and corpus-dependent. Averaging them requires a normalisation constant that becomes wrong the moment the corpus changes. RRF discards magnitudes entirely and uses only rank position, so it is scale-free.

## 3. Citation enforcement

Every factual claim in an answer cites `path:startLine-endLine`.

The generation prompt numbers the retrieved chunks and requires references to them. A post-processing pass then verifies that every citation maps to a chunk that was **actually retrieved**. A citation to something not in context is a hallucination, and it must fail the check loudly rather than being quietly rendered.

## 4. Golden evaluation set — 30 hand-written pairs

You write these yourself, by reading the corpus.

It is tedious. It is also the part that makes you genuinely understand the codebase, and it is exactly what real evaluation work feels like. There is no shortcut that preserves the value.

```
{
  "id": "auth-redirect-01",
  "question": "How does the middleware handle authentication redirects?",
  "expectedSources": ["middleware.ts", "lib/auth/session.ts"],
  "expectedFacts": [
    "redirects to /login",
    "preserves the original URL as a query param"
  ],
  "difficulty": "medium"
}
```

Distribution:

| DIFFICULTY | COUNT | DEFINITION                                                                                |
|------------|-------|-------------------------------------------------------------------------------------------|
| Easy       | 10    | Answer lives in one file                                                                  |
| Medium     | 15    | Requires 2–3 files                                                                        |
| Hard       | 5     | Synthesis across scattered code, <b>or</b> the answer is "this codebase does not do that" |

**Include at least two questions whose correct answer is that the information is absent.** A RAG system that cannot say *"I don't know"* will confidently invent an answer for every gap it has, and you will not discover this unless you test for it.

## 5. Metrics

| METRIC              | WHAT IT MEASURES                                  |
|---------------------|---------------------------------------------------|
| recall@5, recall@10 | Did the right chunks get retrieved at all?        |
| MRR                 | How highly did the best one rank?                 |
| Faithfulness        | Is every claim supported by the retrieved chunks? |
| Citation validity   | Do all cited spans exist in what was retrieved?   |
| p50 / p95 latency   | Ingest and query, measured separately             |

Faithfulness is judged by the local model against a written rubric.

**The judge sees only the generated answer and the retrieved chunks — never the expected answer.** Show it the target and it grades generously. Run it both ways once so you see the effect for yourself, then report the correct configuration.

## Deliverable: `RESULTS.md`

A table comparing baseline against final, and — the part that actually matters — an attribution paragraph.

| CONFIGURATION                          | RECALL@5 | MRR | FAITHFULNESS | P95 QUERY |
|----------------------------------------|----------|-----|--------------|-----------|
| Baseline: 512-char chunks, vector only |          |     |              |           |
| + AST-aware chunking                   |          |     |              |           |
| + BM25 index                           |          |     |              |           |
| + RRF fusion                           |          |     |              |           |
| + deduplication                        |          |     |              |           |

Then, in prose: **which change moved which metric, and why.**

If AST chunking moved recall but not faithfulness, say so and explain it. If deduplication moved nothing, say that — negative results are results, and a change that costs complexity for no measurable gain should be reverted.

"It got better" is not an acceptable conclusion. If you cannot attribute the improvement, you cannot reproduce it, and you were guessing.

## Hard parts you should expect

**Near-duplicate flooding.** Barrel files and re-exports produce near-identical chunks that will occupy your entire top-10. You need deduplication — by content hash first, then by high cosine similarity between retrieved chunks.

**Embedding the query wrong.** `nomic-embed-text` expects task prefixes — `search_query:` for queries, `search_document:` for indexed content. Omitting them costs measurable recall. It is in the model card.

**Slow ingest.** A few hundred files embedded one at a time takes minutes. Batch the embedding calls, and cache by content hash so re-ingest only processes changed files.

**Lost in the middle.** With ten chunks in context, the model reliably underweights the ones in positions four through seven. Ordering matters. Try most-relevant-first against most-relevant-at-the-edges, and measure the difference rather than guessing.

## Deliverables

- ☐ `packages/rag` — ingest, ask, and eval commands
- ☐ AST-aware chunker with the metadata shape above
- ☐ Two indices — `sqlite-vec` and FTS5 — fused with RRF
- ☐ `evals/golden.jsonl` — 30 hand-written pairs
- ☐ Metrics harness reporting all six metrics
- ☐ `RESULTS.md` — the comparison table plus the attribution paragraph
- ☐ `NOTES.md` — why RRF over weighted averaging; the judge-leakage experiment
- ☐ `DESIGN.md` — written **before** the code

## Design doc, before any code

One page, reviewed and approved before you write a line:

1. Public interfaces and types
  2. The three most likely failure modes, and your plan for each
  3. What you are deliberately **not** building, and why
  4. Open questions
- 
- 

## Review gate

Your reviewer will verify these by hand before merge.

- ☐ Delete the vector index. Keyword-only still returns sane results.
- ☐ Delete the FTS index. Vector-only still returns sane results.
- ☐ Ask a question with no answer in the corpus. The system says so and cites nothing.
- ☐ Re-ingest an unchanged corpus. Near-instant, via cache.
- ☐ Every number in `RESULTS.md` reproduces from a fresh clone.